C语言递归详解

一个视频带你浅浅理解一下递归

上面的视频有个特征：自身中又包含了自己，该种思想在数学和编程中非常有用，因为有些时候，我们遇到的问题直接并不好解决，但是发现将原问题拆分成其子问题之后，子问题与原问题有相同的解法，等子问题解决之后，原问题就迎刃而解了。

1. 递归的概念

递归的定义

一个方法在执行过程中调用自身, 就称为 "递归"。

递归相当于数学上的 "数学归纳法", 有一个起始条件, 然后有一个递推公式。

例如, 我们求 N!

起始条件: N = 1 的时候, N! 为 1, 这个起始条件相当于递归的结束条件。

递归公式: 求 N!, 直接不好求, 可以把问题转换成 N! => N * (N-1)!

递归的必要条件:

1. 将原问题划分成其子问题，注意：子问题必须要与原问题的解法相同
2. 递归出口

2. 递归练习

2.1 递归求 N 的阶乘

思路:

1. 先想一下递归的出口，也就是所谓的"起始条件"。
   N == 1的时候，阶乘是1。
2. 递归公式
   5的阶乘 = 5*4的阶乘。N的阶乘=N*(N-1)!

```c
#include <stdio.h>

int fact(int n)
{
    if(n == 0)
    {
        return 1;
    }
    else
    {
        return n * fact(n-1);
    }
}

int main()
{
    int n = 0;
    scanf("%d", &n);
    int ret = fact(n);
    printf("%d\n", ret);
    return 0;
}
```

2.2 递归求1+2+3+4+....+N的和

思路:

1. 先想一下递归的出口，也就是所谓的"起始条件"。
   N==1的时候，和为1。
2. 递归公式
   1+2+3...+N的和 等于 N加1+2+3...+N-1的和。

```c
int sum_n(int n)
{
    if (n == 1)
    {
        return 1;
    }

    return n + sum_n(n - 1);
}
int main()
{
    int ret = sum_n(10);
    printf("%d\n",ret);
    return 0;
}
```

2.3 顺序打印一个整数的每一位

输入一个整数m，按照顺序打印整数的每一位。

比如:

输入：1234　输出：1 2 3 4

输入：520　输出：5 2 0

```c
void print(int n)
{
    if(n > 9)
    {
        print(n/10);
    }
    printf("%d ", n%10);
}

int main()
{
    int m = 0;
    scanf("%d", &m);
    print(m);
    return 0;
}
```

2.4 递归实斐波那契数列的第N项

思路:

1. 先想一下递归的出口，也就是所谓的"起始条件"。
   N==1 或者 N==2 的时候，第N项为1。
2. 递归公式
   第N项 = 第N-1项 + 第N-2项。

```c
int fib(int n)
{
    if (n <= 2)
    {
        return 1;
    }
    else
    {
        return fib(n - 1) + fib(n - 2);
    }
}
int main()
{
    int ret = fib(5);
    printf("%d\n",ret);
    return 0;
}
```

深入思考:

但是这个题不太适合使用递归实现，大家知道为什么吗？（提示：效率问题）

本节总结

- 递归的核心思想：将大问题分解为与原问题解法相同的子问题。
- 递归的两个基本要素：递归出口（终止条件）和递推公式。
- 掌握了阶乘、求和、打印整数、斐波那契数列等经典问题的递归解法。
- 理解了递归函数的设计思路：先确定出口，再确定递推关系。

@博哥C语言制作